



Topic 2: Arrays

Jaber Taheri-Shakib

Research associate

Civil and Environmental Engineering

University of Waterloo | McMaster University

Your Grades Matter



Quick LEARN Question

Please log in to LEARN now and answer the question.

5

minutes



Submit before time ends

1

Open LEARN

Log in and find today's quick question.

2

Answer calmly

This is a positive point only — no negative point.

3

Work quietly

No talking or noise. Stay focused for the full 5 minutes.

Please do not worry — just focus and try your best.

**Participation point
available**

Arrays

- Every programming language has the ability to store sets of related information. The most fundamental way to achieve this is through an array, which is a collection of values under a single variable name, where each element is associated with an index.
- `Grades = [80 75 90 85]; grades(1) is 80.`
- Arrays are typically classified by their dimensions:

Matrix (2D Array)

	Column 1	Column 2	Column 3
Row 1	1	2	3
Row 2	4	5	6
Row 3	7	8	9
Row 4	10	11	12

A 4x3 matrix

`A(2, 3)`

Vector (1D Array)

A 1x4 row vector

2	4	6	8
---	---	---	---

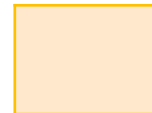
A 3x1
column
vector

2
4
6

Scalar (0D Array)

`x = 5`

A 1x1
scalar



Arrays

MATLAB is unique in that the fundamental unit for the majority of its data types is an array:

```
1  valA = 3;
2  valB = logical(valA);
3  valC = true;
4  valD = "42";
5  valE = '721';

6  valF = valA - valB;
7  valG = valB - valC;
8  valH = valA + double(valD);
9  valI = str2num(valE) - double(valD);
10 valJ = double(valE);
```

Name	Size	Value	Class
valA	1x1	3	double
valB	1x1	1	logical
valC	1x1	1	logical
valD	1x1	"42"	string
valE	1x3	'721'	char
valF	1x1	2	double
valG	1x1	0	double
valH	1x1	45	double
valI	1x1	679	double
valJ	1x3	[55, 50, 49]	double

Vectors

Vectors are commonly used to represent 1D groupings of related data.
For example:



A collection of numerical values associated with the course grades obtained by 5 students.

$grades = [78 \ 85 \ 100 \ 92 \ 96];$



The surface water temperatures at a storm water management pond, sampled over 3 weeks.

$temperature = [21; 23; 22];$

$$\begin{pmatrix} 21 \\ 23 \\ 22 \end{pmatrix}$$

Creating a Row Vector

In MATLAB, the simplest approach to creating a row vector is to enter the vector elements manually using an assignment statement. This is done using the matrix construction operators **[]**.

```
variable_name = [value1 value2 value3 ...]  
or  
variable_name = [value1, value2, value3, ...]
```

variable_name = [comma (or space) separated list of vector elements]

? : How can we create a vector to hold the students' grades?

1	grades = [78 85 100 92 96] Or grades = [78, 85, 100, 92, 96]
---	--

Formally, we have used MATLAB's matrix construction operators to populate a one-dimensional array (i.e. a row vector) containing a collection of students grades under a single variable name (grades).

Automatically Populating a Row Vector

The **colon operator**, `:`, works to populate vectors that have a known increment or spacing.

`variable_name = start:step:last`

- `2:1:4` creates the vector of `[2 3 4]`.
- The default step size is 1. If no `step` is specified, the default will be used. For example `2:4` creates the vector `[2 3 4]` (`2:4` is the same as `2:1:4`)
- Can go in reverse e.g. `4:-1:1` creates `[4 3 2 1]`
(`vec = 4:-1:2`) `vec = [4 3 2]`
- The step size will not jump beyond `last` e.g., `1:2:6` creates `[1 3 5]`
- Will return an empty vector if you provide an instruction that cannot be executed, e.g., `1:-1:10` creates a `1x0` empty row vector (`vec = []`).

How many different way can we create a
?: vector containing the numbers 1 to 4?

`[1 2 3 4]`

```
rvec1 = 1:1:4    vec = [1 2 3 4]
rvec2 = [1, 2, 3, 4]
rvec3 = [1 2 3 4]
rvec4 = 1:4
```

```
rvec5 = 4:-1:1    Output is [4 3 2 1]
rvec6 = [1, 1+1, 1+2, 1+3]
                        Output is [1 2 3 4]
```


Creating a Column Vector

The syntax for manually entering the elements of a column vector is given below.

```
variable_name = [semicolon separated list of vector elements]
```

Example : Suppose we measured surface water temperature over 3 weeks:

1	<code>temp = [21; 23; 22]</code>
---	----------------------------------

temp =
21
23
22

`temp = [21 23 22]`
or
`temp = [21, 23, 22]`
This creates:
1 × 3 row vector
A column vector uses
semicolons:
`temp = [21; 23; 22]`
This creates:
3 × 1 column vector

: The colon operator cannot be used directly to create column vectors. Rather you can create a row vector using the **colon** operator and use matrix operations to replace the row vector with its transpose.

Vector Transpose

The transpose of a matrix is the operation of flipping the matrix over its diagonal (i.e. rows become columns and columns become rows).

$$A = \begin{bmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{bmatrix}_{3 \times 2} \text{ has transpose } A^T = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}_{2 \times 3}$$

Considering a vector as a $1 \times n$ or $n \times 1$ matrix, the transpose operation will flip the vector, making it $n \times 1$ or $1 \times n$, respectively.

```
>> column_vec = [2; 4; 7; 1; 3]
column_vec =
```

```
2
4
7
1
3
```

```
>> row_vec = column_vec'
```

```
row_vec =
```

```
2      4      7      1      3
```

transpose operator

😊: The last element can also be referenced using the keyword `end`.

Accessing Vector Elements

The elements in a vector are numbered sequentially; each element number is called the ***index***, or ***subscript***.

- Consider the vector:

```
myvec = [5 33 11 -4 2];
```

<u>Indices</u>	1	2	3	4	5
<u>Elements</u>	5	33	11	-4	2

Values

Accessing vector elements, also known as referencing or indexing, is achieved by referencing the position, or index, of the element of interest in the vector using round () bracket notation.

value = vector(index)

```
>> value = myvec(3)

value =

    11
```

⚠: The starting index in MATLAB is 1!

Accessing More than One Element

It is often helpful to extract a subset of values from a vector for examination. MatLab accommodates this through index vectors.

`sub_vector = vector(index_vector)`

The index vector:

- Must contain positive integers in the range of the indices of the reference vector
- Can contain the keyword `end`, which is an indirect reference to the last cell in the vector
- Can be populated manually or by using the colon operator (`:`)
- Can be generated in-line or be assigned to a variable which is called in the index expression
- Can include multiple subsets of the reference vector.

```
vec = [10 20 30 40 50];
```

```
sub_vec = vec([1 3 5])
```

```
sub_vec = [10 30 50]
```

```
vec(1:4)
```

```
[10 20 30 40]
```

```
vec(1:2:5)
```

```
[10 30 50]
```

```
vec(end)
```

```
50
```

```
vec(2:end)
```

```
[20 30 40 50]
```

Referencing Example

Consider a row vector containing eight pressure measurements.

1	% ---- vector of experimentally determined pressure measurements (kPa)
2	pressure = [100.34, 123.65, 99.87, 96.45, 98.89, 102.80, 110.39, 95.21];

How can we isolate:

1. The first three elements.
2. The last three elements.
3. The first two and last two elements.
4. Every 3rd element.
5. All elements less than 100 kPa.

1	first3 = pressure(1:3)
2	last3 = pressure(end-2:end)
3	begEnd = pressure([1:2 end-1:end])
4	every3 = pressure(1:3:end)
5	lowP = pressure(pressure < 100)

Name	Value
first3	[100.34 123.65 99.87]
last3	[102.80 110.39 95.21]
begEnd	[100.34 123.65 110.39 95.21]
every3	[100.34 96.45 110.39]
lowP	[99.87 96.45 98.89 95.21]

Common Indexing Errors

Working with arrays can be tricky. You are likely to see the following two errors as you try to work with arrays:

Array indices must be positive integers or logical values.

Index exceeds the number of array elements.

The first error could have been caused by the following commands:

1	value = pressure(0)
2	value = pressure(-1)
3	value = pressure(2.5)

The second error could have been caused by the following commands:

```
pressure = [100.34 123.65 99.87 96.45 98.89 102.80 110.39 95.21];
```

1	value = pressure(10)
2	value = pressure(100)

Modifying Vectors

Now that we can isolate elements in a vector, we can edit them.

variable(index) = expression

A Simple Example

```
rvec = 1:4;  
rvec1 = rvec;  
rvec1(4) = 10;
```

Name	Value
rvec	[1 2 3 4]
rvec1	[1 2 3 10]

: Value(s) computed from expression are assigned to variable at index location(s).

A Complex Example

```
rvec = 1:5;  
rvec1 = rvec;  
rvec2 = rvec;  
rvec3 = rvec;  
  
rvec1([1 3]) = [0 0];  
rvec2(1:3) = 4:6;  
rvec3([1 3]) = rvec3([3 1]);  
rvec4(1:2) = [8, 7, 6];
```

Name	Value
rvec	[1 2 3 4 5]
rvec1	[0 2 0 4 5]
rvec2	[4 5 6 4 5]
rvec3	[3 2 1 4 5]
rvec4	ERROR

: The size of the variable created by the expression must match the size of the referenced subvector!

Concatenation

New vectors can be created by joining existing vectors together, or adding elements to existing vectors. This is called **concatenation**.

[]

cvec2 =

1	<code>rvec = [1 2];</code>
2	<code>rvec1 = [rvec 8 9];</code>
3	<code>rvec2 = [rvec rvec1];</code>
4	<code>rvec(5) = 12;</code>
5	
6	<code>cvec = [4;5];</code>
7	<code>cvec1 = [6;7]</code>
8	<code>cvec2 = [cvec; cvec1];</code>
9	
10	<code>combVec1 = [rvec cvec];</code>
11	<code>combVec2 = [cvec cvec2];</code>

Name	Value
<code>rvec (initial)</code>	<code>[1 2]</code>
<code>rvec1</code>	<code>[1 2 8 9]</code>
<code>rvec2</code>	<code>[1 2 1 2 8 9]</code>
<code>rvec (final)</code>	<code>[1 2 0 0 12]</code>
<code>cvec</code>	<code>[4; 5]</code>
<code>cvec1</code>	<code>[6; 7]</code>
<code>cvec2</code>	<code>[4; 5; 6; 7]</code>
<code>combVec1</code>	ERROR
<code>combVec2</code>	ERROR

4
5
6
7



Math With Vectors

MatLab allows you to perform mathematical operations on vectors using *element-wise* operators and *array* operators.

- Element-wise operators perform operations individually on each element of the vector.

These operators include:

- + addition
- subtraction
- dot operator (couples with other operators such as *, /, ^ to force element-wise behavior)

a = [1 2 3]; b = [4 5 6]; If we add them: c = a + b	MATLAB calculates: [1+4 2+5 3+6] So the answer is: c = [5 7 9]
--	---

- Array Operators perform matrix computations, such as matrix multiplication and inversion. These operators include:

- * multiplication
- / inversion (*we'll talk about this one more in depth later*)

⚠: Element-wise operations can only be completed between vectors of *equivalent* shape.

⚠: Array operations can only be completed between vectors of *compatible* shape.

a .* b
means:
[1*4 2*5 3*6]
So the result is:
[4 10 18]

a.^2
means square each element:
[1^2 2^2 3^2]
So the result is:
[1 4 9]

a = [1 2 3];
b = [4 5];
a + b
ERROR

Element-Wise Operations

```
1 rvec1 = [6, 0, 2, 5];
2 rvec2 = [9, 9, 1, 9];
3 cvec = [9; 4; 8; 1];
4
5 fprintf('Addition:\n')
6 disp(rvec1+rvec2)
7
8 fprintf('Subtraction:\n')
9 disp(rvec1 - 3)
10
11 fprintf('Division:\n')
12 disp(rvec1./rvec2)
13
14 fprintf('Addition Part 2:\n')
15 disp(rvec1+cvec)
```

[6+9 0+9 2+1 5+9]

[6-3 0-3 2-3 5-3]

[6/9 0/9 2/1 5/9]

9 + [6 0 2 5] = [15 9 11 14]
4 + [6 0 2 5] = [10 4 6 9]
8 + [6 0 2 5] = [14 8 10 13]
1 + [6 0 2 5] = [7 1 3 6]

Addition:

15 9 3 14

Subtraction:

3 -3 -1 2

Division:

0.6667 0 2.0000 0.5556

Addition Part 2:

15 9 11 14

10 4 6 9

14 8 10 13

7 1 3 6

Array Operations

```
1 rvec1 = [6, 0, 2, 5];
2 rvec2 = [9, 9, 1, 9];
3 cvec1 = [9; 4; 8; 1];
4 cvec2 = [4; 3; 8; 6];
5
6 fprintf('Col*Row:\n')
7 disp(cvec1*rvec1)
8
9 fprintf('Row*Col:\n')
10 disp(rvec1*cvec1)
11
12 fprintf('Row*Row:\n')
13 disp(rvec1*rvec2)
14
15 fprintf('Col*Col:\n')
16 disp(cvec1*cvec2)
```

Col*Row:

54	0	18	45
24	0	8	20
48	0	16	40
6	0	2	5

Row*Col:

75

Row*Row:

ERROR

Col*Col:

ERROR

Functions and Arrays

- As MatLab is designed for arrays, the majority of its functions support array arguments or are designed specifically for use with arrays.
- All of the functions we've learned so far will perform element-wise operations on input arrays. For example:

```
1  vec = [6.2, 0.11, 2.6, 5.3];  
2  vecOut1 = exp(vec);  
3  vecOut2 = round(vec);  
4  vecOut3 = sin(vec);  
5  vecOut4 = sqrt(vec);
```

exp(6.2), exp(0.11), exp(2.6), exp(5.3)

round([6.2, 0.11, 2.6, 5.3])

radians

sqrt([6.2, 0.11, 2.6, 5.3])

Name	Value
vec	[6.2 0.11 2.6 5.3]
vecOut1	[492.75 1.12 13.46 200.34]
vecOut2	[6 0 3 5]
vecOut3	[-0.08 0.11 0.52 -0.83]
vecOut4	[2.49 0.33 1.61 2.30]

Vector Functions

Function	Description	Example	
		Script	Command Window
<code>linspace(x1,x2,n)</code>	creates a vector of n points linearly spaced from x1 to x2	<code>linspace(0,2,3)</code>	<code>[0,1,2]</code>
<code>length(A)</code>	Returns the number of elements in A	<code>length([1,2,3,4])</code>	<code>4</code>
<code>sum(A)</code>	Computes the sum of all the elements in A	<code>sum([1 2 3 4])</code>	<code>10</code>
<code>mean(A)</code>	Computes the mean value of all the elements in A	<code>mean([1 2 3 4])</code>	<code>2.5</code>
<code>median(A)</code>	Returns the median value of the elements in A	<code>median([7 1 4])</code>	<code>4</code>
<code>std(A)</code>	Computes the N-1 normalized standard deviation of the elements in A , where N is the number of elements.	<code>std([1 2 3 4])</code> 	<code>1.29</code>
<code>min(A)</code>	Returns the smallest element of A	<code>min([9 -2 0 4])</code>	<code>-2</code>
<code>[val, ind] = min(A)</code>	Returns the value and index of the minimum	<code>[val,ind] = min([9 -2 0 4])</code>	<code>[-2,2]</code>
<code>max(A)</code>	Returns the largest element of A	<code>max([9 -2 0 4])</code>	<code>9</code>
<code>[val, ind] = max(A)</code>	Returns the value and index of the maximum	<code>[val,ind] = max([9 -2 0 4])</code>	<code>[9,1]</code>

Printing Out Vectors

- When a vector is used as an argument for the `fprintf()` function, the output is printed repeatedly for every element in the vector.
- We can use this to our advantage to quickly display vectors in a coherent way.

```
1  vec = [6.2, 0.11, 2.6, 5.3];
2
3  % Option 1
4  fprintf('Vector Elements:\n')
5  fprintf('\t%.2f\n', vec)
6
7  % Option 2
8  fprintf('The output vector is [')
9  fprintf('%.2f  ', vec)
10 fprintf('].\n')
11
12 % ... or any other combination to
13 % effectively communicate your output.
```

Vector Elements:

6.20

0.11

2.60

5.30

The output vector is [6.20 0.11
2.60 5.30].

Question: Groundwater Quality Monitoring

A geotechnical/environmental engineer measured nitrate concentration in groundwater samples from 8 monitoring wells near a landfill site.

The nitrate concentration values are given in mg/L:

nitrate = [6.5 8.2 10.5 4.8 12.3 7.1 9.6 11.8];

The safe drinking-water limit for nitrate is:

limit = 10;

Tasks

Write a MATLAB script to:

1. Create the vector *nitrate*.
2. Display the first three nitrate measurements.
3. Display the last two nitrate measurements.
4. Calculate and display the average nitrate concentration.
5. Find the maximum nitrate concentration and its well number.
6. Extract and display all nitrate values greater than the safe limit.
7. Calculate how much each measurement is above or below the safe limit using: *difference* = *nitrate* - *limit*;
8. Print the results clearly using `fprintf`.

Question: Groundwater Quality Monitoring

```
clear
clc
```

```
% Groundwater nitrate concentration data in mg/L
nitrate = [6.5 8.2 10.5 4.8 12.3 7.1 9.6 11.8];
```

```
% Safe limit in mg/L
limit = 10;
```

```
% 1. First three measurements
first3 = nitrate(1:3);
```

```
% 2. Last two measurements
last2 = nitrate(end-1:end);
```

```
% 3. Average nitrate concentration
avgNitrate = mean(nitrate);
```

```
% 4. Maximum nitrate concentration and well number
[maxNitrate, wellNumber] = max(nitrate);
```

```
% 5. Values greater than safe limit
highNitrate = nitrate(nitrate > limit);
```

```
% 6. Difference from safe limit
difference = nitrate - limit;
```

```
% Display results
fprintf('First three nitrate measurements: ')
fprintf('%0.1f ', first3)
fprintf('\n')
```

```
fprintf('Last two nitrate measurements: ')
fprintf('%0.1f ', last2)
fprintf('\n')
```

```
fprintf('Average nitrate concentration = %0.2f mg/L\n', avgNitrate)
```

```
fprintf('Maximum nitrate concentration = %0.1f mg/L\n', maxNitrate)
fprintf('This occurred at well number %d.\n', wellNumber)
```

```
fprintf('Nitrate values above the safe limit: ')
fprintf('%0.1f ', highNitrate)
fprintf('\n')
```

```
fprintf('Difference from safe limit: ')
fprintf('%0.1f ', difference)
fprintf('\n')
```

Question: Groundwater Quality Monitoring

Expected Output

First three nitrate measurements: 6.5 8.2 10.5

Last two nitrate measurements: 9.6 11.8

Average nitrate concentration = 8.85 mg/L

Maximum nitrate concentration = 12.3 mg/L

This occurred at well number 5.

Nitrate values above the safe limit: 10.5 12.3 11.8

Difference from safe limit: -3.5 -1.8 0.5 -5.2 2.3 -2.9 -0.4 1.8